

Project Proposal: Pulsar

A Decoupled Distributed Publish-Subscribe System

March 21, 2026

1 Problem Statement

Traditional messaging systems often tightly couple message processing (compute) with message persistence (storage) on the same nodes, leading to complex scaling and fault-tolerance challenges. Apache Pulsar solves this by decoupling the broker layer from the storage layer.

The objective of this project is to design and develop “Mini-Pulsar,” a distributed publish-subscribe messaging system that mimics this multi-tier architecture. The system will enable asynchronous, topic-based message streaming between producers and consumers. By building this from the ground up, the project will practically demonstrate core distributed system concepts including RPC-based node communication, distributed consensus, fault tolerance, and custom log-based message persistence.

2 Proposed Architecture and Scope

To ensure the project is deliverable within the timeframe while fulfilling distributed systems learning outcomes, the scope will focus on a localized cluster simulation featuring the following decoupled components:

- **Clients (Producers & Consumers):** Producers will publish events to specific distributed topics. Consumers will subscribe to these topics, demonstrating a fan-out message delivery pattern where multiple subscribers receive real-time messages.
- **Broker Nodes (Compute Tier):** Stateless servers responsible for routing messages. Brokers will accept incoming RPC calls from producers, route the data to the storage tier, and push messages to connected consumers.
- **Storage Nodes (Data Tier):** Instead of an external database, the system will utilize a custom-built, append-only log mechanism on the local file system. This will simulate Pulsar’s “BookKeeper” nodes, ensuring message persistence and strict ordering guarantees.
- **Metadata Coordinator (Consensus Tier):** A localized implementation of a consensus algorithm (e.g., a simplified Raft) to manage the cluster’s state. This component will handle broker leader election, maintain the registry of topics, and track which brokers are responsible for which partitions.

3 Replicating Pulsar’s Core Functionalities

The system will implement the following Pulsar-specific mechanics leveraging the decoupled architecture:

- **Load Balancing:** Because the Broker nodes are stateless, the Metadata Coordinator can seamlessly reassign topic partitions to different Brokers during high traffic or node failures without needing to migrate any underlying data.
- **Durable Subscriptions:** The system will implement a cursor-tracking mechanism. Brokers will log consumer acknowledgments (ACKs) as offsets. If a consumer disconnects, the system will retain unacknowledged messages and resume delivery from the exact saved cursor upon reconnection.
- **Message Retention Policies:** Storage Nodes will feature a background process to enforce retention limits (based on file size or timestamps). Once a log segment exceeds the policy limits, it will be automatically truncated or deleted from the local disk to manage storage efficiently.

4 Frameworks and Technologies

- **Programming Language:** Python, chosen for its rapid development capabilities and robust standard library for file I/O and threading.
- **Communication Protocol:** gRPC and Protocol Buffers (Protobuf). gRPC will be heavily utilized for all inter-node communication, including client-to-broker messaging, broker-to-storage persistence calls, and coordinator heartbeats.
- **Storage:** Custom Python file-handling to create immutable, append-only logs representing message topics.
- **Testing Environment:** Local multi-process execution simulating a distributed cluster environment.

5 Deliverables

The project will be evaluated based on the successful local execution of the distributed components and the architectural decisions made during development. The final deliverables will include:

1. **Source Code:** A fully commented codebase containing the implementations for the Clients, Brokers, Storage Nodes, and the Metadata Coordinator.
2. **Working Prototype:** A functional, locally simulated cluster demonstrating live message publishing, fan-out consumption, durable subscriptions, and fault tolerance.
3. **Architecture and Design Document:** A comprehensive document detailing the system architecture, the custom log-based storage design, the gRPC service definitions, and an analysis of the consensus algorithm's role in the system.
4. **Execution Guide (README):** Step-by-step instructions to spin up the local cluster, initialize topics, and run producer/consumer test scripts to verify the system's behavior.
5. **Final Project Report:** A retrospective report analyzing system performance, bottlenecks, and the theoretical guarantees (e.g., CAP theorem tradeoffs) achieved by the architecture.